Formation 11 : Pytorch

KRYPTOSPHERE®

```
alexandre@PCAlexandre  ~  pip install torch torchvision
```

Installation des bibliothèques

Importer les modules

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

from torch.utils.data import DataLoader, TensorDataset, random_split
from torchvision import datasets, transforms
from sklearn.metrics import f1_score
```

Création d’un modèle

```python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def create_model():
    return nn.Sequential(
        nn.Linear("taille_entrée", "taille_sortie"),
        nn.ReLU(),
        nn.Dropout("proportion à drop"),
        nn.Linear("taille_entrée", "taille_sortie")
    )

model = create_model().to(device)
print(model)
print(f"Device utilisé : {device}")
```

Paramètres du modèle

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

```python
full_train_dataset = TensorDataset(x_train, y_train)
train_size = int(0.9 * len(full_train_dataset))
val_size = len(full_train_dataset) - train_size
train_dataset_split, val_dataset_split = random_split(full_train_dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset_split, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset_split, batch_size=32, shuffle=False)
```

Le cœur de l’entrainement

```python
history = {"loss": [], "val_loss": [], "accuracy": [], "val_accuracy": []}
epochs = 10

for epoch in range(epochs):
    model.train()
    train_loss = 0.0
    train_correct = 0
    train_total = 0

    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

        train_loss += loss.item() * xb.size(0)
        preds = logits.argmax(dim=1)
        train_correct += (preds == yb).sum().item()
        train_total += yb.size(0)

    train_loss /= train_total
    train_acc = train_correct / train_total

    model.eval()
    val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)

            val_loss += loss.item() * xb.size(0)
            preds = logits.argmax(dim=1)
            val_correct += (preds == yb).sum().item()
            val_total += yb.size(0)

    val_loss /= val_total
    val_acc = val_correct / val_total

    history["loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["accuracy"].append(train_acc)
    history["val_accuracy"].append(val_acc)

    print(f"Epoch {epoch + 1}/{epochs} - loss: {train_loss:.4f} - accuracy: {train_acc:.4f} - val_loss: {val_loss:.4f} - val_accuracy: {val_acc:.4f}")
```

```python
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=32, shuffle=False)

model.eval()
test_loss = 0.0
test_correct = 0
test_total = 0

with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = criterion(logits, yb)

        test_loss += loss.item() * xb.size(0)
        preds = logits.argmax(dim=1)
        test_correct += (preds == yb).sum().item()
        test_total += yb.size(0)

test_loss /= test_total
test_accuracy = test_correct / test_total

print(f"Test loss     : {test_loss:.4f}")
print(f"Test accuracy : {test_accuracy:.4f}")
```

Evaluation du modèle